

E2

E2: DEMO - Time Difference between subsequent Positions

Difftime function: Assume we want to calculate the timelag between subsequent positions. To achieve this we can use the function `difftime()` combined with `lead()` from `dplyr`. Let's look at thgit initese functions one by one.

- Distanz zwischen zwei Zeitpunkten im Posixct- Format. Hier werden die Zeitpunkte `later` und `now` genannt und mit `difftime` die Unterschiede berechnet.
- Man wandelt es meist in numeric um, damit es einfacher ist, zb in 4500s. Das macht man mit der Funktion `difftime_secs()`.

```
#difftime takes two POSIXct values.
```

```
now <- as.POSIXct("2024-04-26 10:20:00")
later <- as.POSIXct("2024-04-26 11:35:00")

later
```

```
[1] "2024-04-26 11:35:00 CEST"
```

```
time_difference <- difftime(later, now)
time_difference
```

Time difference of 1.25 hours

```
#You can also specify the unit of the output.
time_difference <- difftime(later, now, units = "secs")
time_difference
```

Time difference of 4500 secs

```
#difftime returns an object of the class difftime.  
class(time_difference)
```

```
[1] "difftime"
```

```
## [1] "difftime"  
  
str(time_difference)
```

```
'difftime' num 4500  
- attr(*, "units")= chr "secs"
```

```
## 'difftime' num 4500  
## - attr(*, "units")= chr "secs"
```

However in our case, numeric values would be more handy than the class difftime. So we'll wrap the command in as.numeric():

```
time_difference <- as.numeric(difftime(later, now, units = "secs"))  
  
str(time_difference)
```

```
num 4500
```

```
## num 4500  
class(time_difference)
```

```
[1] "numeric"
```

```
## [1] "numeric"
```

In fact, we will use this exact operation multiple times, so let's create a function for this:

```
difftime_secs <- function(later, now){  
  as.numeric(difftime(later, now, units = "secs"))  
}
```

Lead() /Lag()

`lead()` and `lag()` return a vector of the same length as the input, just offset by a specific number of values (default is 1). Consider the following sequence.

We can now run `lead()` and `lag()` on this sequence to illustrate the output. `n` = specifies the offset, `default` = specifies the default value used to “fill” the emerging “empty spaces” of the vector. This helps us performing operations on subsequent values in a vector (or rows in a table).

- Werte innerhalb eines Vektors oder einer Spalte zeitlich bzw. positionsmässig zu verschieben. Sie geben die gleiche Länge zurück wie ursprünglich, aber verschoben.
- `lag()` greift auf vorherige Werte zu
- `Lead()` greift auf nachfolgende Werte zu
- `n` = wie viele Positionen werden verschoben? `default`= Welcher Wert die entstehenden Lücken füllt, zb NA oder hier 1?)
- Was nützt es uns? man kann Beziehungen zwischen aufeinanderfolgenden Daten herstellen, ohne dass man Schleifen schreiben muss.

Dieses Beispiel 1-10. Bei `Lead` wird das vorherige geholt—> hier NA am Schluss. Bei `Lead 2` mal wird 2 mal die vorherigen geholt, also NA zu Beginn. Den `Default` kann man auch ändern, dann zeigt es kein NA, sondern 0.

```
numbers <- 1:10  
numbers
```

```
[1]  1  2  3  4  5  6  7  8  9 10
```

```
library("dplyr")
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

```
filter, lag
```

The following objects are masked from 'package:base':

```
intersect, setdiff, setequal, union
```

```
lead(numbers)
```

```
[1]  2  3  4  5  6  7  8  9 10 NA
```

```
lead(numbers, n = 2)
```

```
[1]  3  4  5  6  7  8  9 10 NA NA
```

```
lag(numbers)
```

```
[1] NA  1  2  3  4  5  6  7  8  9
```

```
lag(numbers, n = 5)
```

```
[1] NA NA NA NA NA  1  2  3  4  5
```

```
lag(numbers, n = 5, default = 0)
```

```
[1] 0 0 0 0 0 1 2 3 4 5
```

Beim Übergang vom einen Tier ins andere (Hans und Klara) wird ein falscher Wert entstehen, da wir nicht gruppiert haben (ein negativer Zeitunterschied), deshalb gruppieren wird das Ganze dann noch.

mutate()

```
wildschwein <- tibble(      # aka data.frame
  TierID = rep(c("Hans", "Klara"), each = 5),
  DatetimeUTC = rep(as.POSIXct("2015-01-01 00:00:00", tz = "UTC") + 0:4 * 15 * 60, 2)
)

wildschwein
```

```
# A tibble: 10 x 2
  TierID DatetimeUTC
  <chr>   <dtm>
1 Hans   2015-01-01 00:00:00
2 Hans   2015-01-01 00:15:00
3 Hans   2015-01-01 00:30:00
4 Hans   2015-01-01 00:45:00
5 Hans   2015-01-01 01:00:00
6 Klara  2015-01-01 00:00:00
7 Klara  2015-01-01 00:15:00
8 Klara  2015-01-01 00:30:00
9 Klara  2015-01-01 00:45:00
10 Klara 2015-01-01 01:00:00
```

Speed berechnen zwischen zwei Zeitpunkten

```
now <- wildschwein$DatetimeUTC
later <- lead(now)

# View(wildschwein)

wildschwein$timelag <- difftime_secs(later, now)

wildschwein
```

```
# A tibble: 10 x 3
  TierID DatetimeUTC      timelag
  <chr>   <dtm>         <dbl>
1 Hans   2015-01-01 00:00:00      900
2 Hans   2015-01-01 00:15:00      900
3 Hans   2015-01-01 00:30:00      900
4 Hans   2015-01-01 00:45:00      900
5 Hans   2015-01-01 01:00:00    -3600
6 Klara  2015-01-01 00:00:00      900
7 Klara  2015-01-01 00:15:00      900
8 Klara  2015-01-01 00:30:00      900
9 Klara  2015-01-01 00:45:00      900
10 Klara 2015-01-01 01:00:00      NA
```

```
#Mit mutate kein dollarzeichen: note the lack of "$"
wildschwein <- mutate(wildschwein, timelag = difftime_secs(lead(DatetimeUTC), DatetimeUTC))
```

```
wildschwein
```

```
# A tibble: 10 x 3
  TierID DatetimeUTC      timelag
  <chr>   <dtm>         <dbl>
1 Hans   2015-01-01 00:00:00      900
2 Hans   2015-01-01 00:15:00      900
3 Hans   2015-01-01 00:30:00      900
4 Hans   2015-01-01 00:45:00      900
5 Hans   2015-01-01 01:00:00    -3600
6 Klara   2015-01-01 00:00:00      900
7 Klara   2015-01-01 00:15:00      900
8 Klara   2015-01-01 00:30:00      900
9 Klara   2015-01-01 00:45:00      900
10 Klara  2015-01-01 01:00:00      NA
```

group_by()

Jetzt wird der letzte Wert pro Tier dann als NA angezeigt, weil eine Berechnung des Übergangs zwischen den Tieren keinen Sinn ergibt.

```
# again, note the lack of "$"
wildschwein <- group_by(wildschwein, TierID)
```

```
# again, note the lack of "$"
wildschwein <- mutate(wildschwein, timelag = difftime_secs(lead(DatetimeUTC), DatetimeUTC))

wildschwein
```

```
# A tibble: 10 x 3
# Groups:   TierID [2]
  TierID DatetimeUTC      timelag
  <chr>   <dtm>         <dbl>
1 Hans   2015-01-01 00:00:00      900
2 Hans   2015-01-01 00:15:00      900
3 Hans   2015-01-01 00:30:00      900
4 Hans   2015-01-01 00:45:00      900
5 Hans   2015-01-01 01:00:00      NA
6 Klara   2015-01-01 00:00:00      900
7 Klara   2015-01-01 00:15:00      900
```

```

8 Klara 2015-01-01 00:30:00 900
9 Klara 2015-01-01 00:45:00 900
10 Klara 2015-01-01 01:00:00 NA

```

Piping

So geht es noch einfacher mit Piping:

```

wildschwein |>                                # Take wildschwein...
  group_by(TierID) |>                          # ...group it by TierID
  mutate(
    timelag = difftime_secs(lead(DatetimeUTC), DatetimeUTC) # Caculate difftime
  )

```

```

# A tibble: 10 x 3
# Groups:   TierID [2]
  TierID DatetimeUTC      timelag
  <chr>   <dtm>         <dbl>
1 Hans   2015-01-01 00:00:00    900
2 Hans   2015-01-01 00:15:00    900
3 Hans   2015-01-01 00:30:00    900
4 Hans   2015-01-01 00:45:00    900
5 Hans   2015-01-01 01:00:00     NA
6 Klara  2015-01-01 00:00:00    900
7 Klara  2015-01-01 00:15:00    900
8 Klara  2015-01-01 00:30:00    900
9 Klara  2015-01-01 00:45:00    900
10 Klara 2015-01-01 01:00:00     NA

```

summarise()

```

summarise(wildschwein, mean = mean(timelag, na.rm = TRUE))

```

```

# A tibble: 2 x 2
  TierID mean
  <chr>   <dbl>
1 Hans    900
2 Klara   900

```

E2: Exercise A

Task 1: Import your Data

& Data Frame daraus machen.

- es ist bereits im richtigen Koordinatensystem EPSG 2056
- Coordinates sind stored in N und E columns.
- Sabi, Rosa, Ruth sind die Namen der Tiere
- Man muss das Zeitformat als Posixct machen

```
library("readr")  
library("sf")
```

Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE

```
library("tidyverse")
```

Warning: package 'tibble' was built under R version 4.5.2

Warning: package 'tidyr' was built under R version 4.5.2

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v forcats   1.0.1      v stringr   1.5.2  
v ggplot2   4.0.0      v tibble    3.3.1  
v lubridate 1.9.4      v tidyr     1.3.2  
v purrr     1.1.0
```

```
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library("dplyr")
```

```
wildschwein_BE <- read_delim("Data/wildschwein_BE_2056.csv", ",",
```


Rows: 51246 Columns: 6

```
-- Column specification -----  
Delimiter: ","  
chr  (2): TierID, TierName  
dbl  (3): CollarID, E, N  
dtm  (1): DatetimeUTC
```

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
wildschwein_BE <- st_as_sf(wildschwein_BE, coords = c("E", "N"), crs = 2056)  
  
head(wildschwein_BE)
```

Simple feature collection with 6 features and 4 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570379 ymin: 1204752 xmax: 2570409 ymax: 1204863

Projected CRS: CH1903+ / LV95

A tibble: 6 x 5

	TierID	TierName	CollarID	DatetimeUTC	geometry
	<chr>	<chr>	<dbl>	<dtm>	<POINT [m]>
1	002A	Sabi	12275	2014-08-22 21:00:12	(2570409 1204752)
2	002A	Sabi	12275	2014-08-22 21:15:16	(2570402 1204863)
3	002A	Sabi	12275	2014-08-22 21:30:43	(2570394 1204826)
4	002A	Sabi	12275	2014-08-22 21:46:07	(2570379 1204817)
5	002A	Sabi	12275	2014-08-22 22:00:22	(2570390 1204818)
6	002A	Sabi	12275	2014-08-22 22:15:10	(2570390 1204825)

```
str(wildschwein_BE)
```

```
sf [51,246 x 5] (S3: sf/tbl_df/tbl/data.frame)
```

```
$ TierID      : chr [1:51246] "002A" "002A" "002A" "002A" ...
```

```
$ TierName    : chr [1:51246] "Sabi" "Sabi" "Sabi" "Sabi" ...
```

```
$ CollarID    : num [1:51246] 12275 12275 12275 12275 12275 ...
```

```
$ DatetimeUTC: POSIXct[1:51246], format: "2014-08-22 21:00:12" "2014-08-22 21:15:16" ...
```

```
$ geometry    :sfc_POINT of length 51246; first list element: 'XY' num [1:2] 2570409 1204752
```

```
- attr(*, "sf_column")= chr "geometry"
```

```
- attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA
```

```
..- attr(*, "names")= chr [1:4] "TierID" "TierName" "CollarID" "DatetimeUTC"
```

```
wildschwein_BE$DatetimeUTC <- as.POSIXct(wildschwein_BE$DatetimeUTC, tz = "UTC")
```

Task 2: Getting an Overview

Calculate the time difference between subsequent rows as described in the demo with `lead()` and `difftime_secs()` in a new column `timelag`.

Du solltest die Zeitdifferenz **pro Tier (TierID)** berechnen und die Daten vorher **zeitlich sortieren**.

- `arrange`: Reihenfolge bestimmen
- `group by` nach `TierID`
- `timelag`: Differenz in sekunden -> `timelag` Spalte

```
difftime_secs <- function(later, now){  
  as.numeric(difftime(later, now, units = "secs"))  
}  
  
wildschwein_BE <- wildschwein_BE %>%  
  arrange(TierID) %>% # nach Tier  
  mutate(  
    timelag = difftime_secs(lead(DatetimeUTC), DatetimeUTC)  
  )
```

- How many individuals were tracked? 3 Tiere
- For how long were the individual tracked? Are there gaps? Zu Unterschiedlichen Zeiten
- Were all individuals tracked concurrently or sequentially? also überlappend oder nacheinander? (überlappend: 2014 und 2015)
- What is the temporal sampling interval between the locations? 15Min

```
library("tidyverse")  
wildschwein_BE
```

Simple feature collection with 51246 features and 5 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2568153 ymin: 1202306 xmax: 2575154 ymax: 1207609

Projected CRS: CH1903+ / LV95

A tibble: 51,246 x 6

```

  TierID TierName CollarID DatetimeUTC          geometry
* <chr>  <chr>          <dbl> <dtm>          <POINT [m]>
1 002A   Sabi           12275 2014-08-22 21:00:12 (2570409 1204752)
2 002A   Sabi           12275 2014-08-22 21:15:16 (2570402 1204863)
3 002A   Sabi           12275 2014-08-22 21:30:43 (2570394 1204826)
4 002A   Sabi           12275 2014-08-22 21:46:07 (2570379 1204817)
5 002A   Sabi           12275 2014-08-22 22:00:22 (2570390 1204818)
6 002A   Sabi           12275 2014-08-22 22:15:10 (2570390 1204825)
7 002A   Sabi           12275 2014-08-22 22:30:13 (2570387 1204831)
8 002A   Sabi           12275 2014-08-22 22:45:11 (2570381 1204840)
9 002A   Sabi           12275 2014-08-22 23:00:27 (2570316 1204935)
10 002A  Sabi           12275 2014-08-22 23:15:41 (2570393 1204815)
# i 51,236 more rows
# i 1 more variable: timelag <dbl>

```

```
length(unique(wildschwein_BE$TierName))
```

```
[1] 3
```

```

wildschwein_BE |>
  group_by(TierID) |>
  summarise(
    start = min(DatetimeUTC),
    end = max(DatetimeUTC),
    duration = max(DatetimeUTC) - min(DatetimeUTC)
  )

```

Simple feature collection with 3 features and 4 fields

Geometry type: MULTIPOINT

Dimension: XY

Bounding box: xmin: 2568153 ymin: 1202306 xmax: 2575154 ymax: 1207609

Projected CRS: CH1903+ / LV95

A tibble: 3 x 5

```

  TierID start          end          duration
  <chr>  <dtm>          <dtm>          <drtn>
1 002A   2014-08-22 21:00:12 2015-07-27 11:00:14 338.5834 days
2 016A   2014-11-07 07:45:44 2015-06-29 23:45:11 234.6663 days
3 018A   2014-11-07 18:00:43 2015-07-27 09:45:15 261.6559 days
# i 1 more variable: geometry <MULTIPOINT [m]>

```

```
wildschwein_BE %>%
  st_drop_geometry() %>%
  group_by(TierName) %>%
  summarise(duration_sec = sum(timelag, na.rm = TRUE),
            duration_h = sum(timelag, na.rm = TRUE)/3600,
            from = min(DatetimeUTC, na.rm = TRUE),
            to = max(DatetimeUTC, na.rm = TRUE))
```

```
# A tibble: 3 x 5
  TierName duration_sec duration_h from to
  <chr>      <dbl>      <dbl> <dtm> <dtm>
1 Rosa      36899      10.2 2014-11-07 07:45:44 2015-06-29 23:45:11
2 Ruth     22607072     6280. 2014-11-07 18:00:43 2015-07-27 09:45:15
3 Sabi      6605132     1835. 2014-08-22 21:00:12 2015-07-27 11:00:14
```

```
library(dplyr)

wildschwein_BE %>%
  slice_max(timelag, n = 1)
```

Simple feature collection with 1 feature and 5 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2575073 ymin: 1203209 xmax: 2575073 ymax: 1203209

Projected CRS: CH1903+ / LV95

```
# A tibble: 1 x 6
  TierID TierName CollarID DatetimeUTC geometry timelag
  <chr> <chr>      <dbl> <dtm> <POINT [m]> <dbl>
1 016A Rosa      13972 2015-04-18 02:46:38 (2575073 1203209) 60367
```

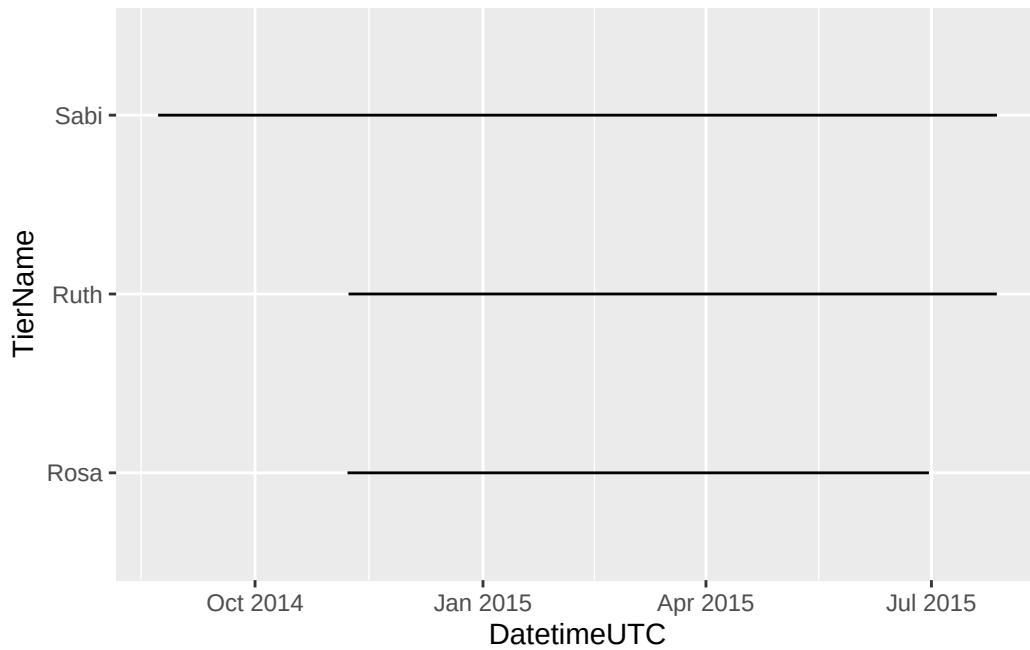
```
wildschwein_BE %>%
  st_drop_geometry() %>%
  group_by(TierName) %>%
  summarise(sum_gaps_1h = sum(timelag > 3600, na.rm = TRUE),
            sum_gaps_2h = sum(timelag > 7200, na.rm = TRUE)) # Anzahl Zeitschritte > 1h
```

```
# A tibble: 3 x 3
  TierName sum_gaps_1h sum_gaps_2h
  <chr>      <int>      <int>
1 Rosa      938        408
```

2	Ruth	1147	523
3	Sabi	1138	542

```
library(tidyverse)

ggplot(wildschwein_BE, aes(x= DatetimeUTC, y = TierName)) +
  geom_line()
```



```
paste("Im Durchschnitt", round(mean(wildschwein_BE$timelag, na.rm = TRUE)/60, 0), "Minuten")
```

```
[1] "Im Durchschnitt 10 Minuten"
```

```
paste("Im Median", round(median(wildschwein_BE$timelag, na.rm = TRUE)/60, 0), "Minuten")
```

```
[1] "Im Median 15 Minuten"
```

Task 3: Distance between locations

```

later <- lag(wildschwein_BE$geometry)
now <- wildschwein_BE$geometry

distance<-st_distance(later, now, by_element = TRUE) # by_element must be set to TRUE

distance_by_element <- function(later, now){
  as.numeric(
    st_distance(later, now, by_element = TRUE)
  )
}

```

```

wildschwein_BE$steplength <- distance_by_element(later, now)
head(wildschwein_BE)

```

Simple feature collection with 6 features and 6 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570379 ymin: 1204752 xmax: 2570409 ymax: 1204863

Projected CRS: CH1903+ / LV95

A tibble: 6 x 7

	TierID	TierName	CollarID	DatetimeUTC	geometry	timelag
	<chr>	<chr>	<dbl>	<dtm>	<POINT [m]>	<dbl>
1	002A	Sabi	12275	2014-08-22 21:00:12	(2570409 1204752)	904
2	002A	Sabi	12275	2014-08-22 21:15:16	(2570402 1204863)	927
3	002A	Sabi	12275	2014-08-22 21:30:43	(2570394 1204826)	924
4	002A	Sabi	12275	2014-08-22 21:46:07	(2570379 1204817)	855
5	002A	Sabi	12275	2014-08-22 22:00:22	(2570390 1204818)	888
6	002A	Sabi	12275	2014-08-22 22:15:10	(2570390 1204825)	903

i 1 more variable: steplength <dbl>

speed Berechnen

aus steplength and the timelag (from the last task).

```

wildschwein_BE$speed <- round(wildschwein_BE$steplength/wildschwein_BE$timelag, 4)
head(wildschwein_BE)

```

Simple feature collection with 6 features and 7 fields

```

Geometry type: POINT
Dimension:      XY
Bounding box:   xmin: 2570379 ymin: 1204752 xmax: 2570409 ymax: 1204863
Projected CRS: CH1903+ / LV95
# A tibble: 6 x 8
  TierID TierName CollarID DatetimeUTC          geometry timelag
  <chr>   <chr>      <dbl> <dtm>          <POINT [m]>    <dbl>
1 002A   Sabi        12275 2014-08-22 21:00:12 (2570409 1204752)    904
2 002A   Sabi        12275 2014-08-22 21:15:16 (2570402 1204863)    927
3 002A   Sabi        12275 2014-08-22 21:30:43 (2570394 1204826)    924
4 002A   Sabi        12275 2014-08-22 21:46:07 (2570379 1204817)    855
5 002A   Sabi        12275 2014-08-22 22:00:22 (2570390 1204818)    888
6 002A   Sabi        12275 2014-08-22 22:15:10 (2570390 1204825)    903
# i 2 more variables: steplength <dbl>, speed <dbl>

```

Visualize / Plausibility Check

Immer anschauen, ob es Sinn ergibt. Dann print und save als Bild.

```

wildschwein_sample <- wildschwein_BE %>%
  filter(TierName == "Sabi") %>%
  head(100)

library(tmap)
tmap_mode("view")

```

```

i tmap modes "plot" - "view"
i toggle with `tmap::ttm()`

```

```

m <- tm_shape(wildschwein_sample) +
  tm_dots()

print(tmap_leaflet(m))
tmap_save(m, "Week_2_Karte_Linie.html")

```

Interactive map saved to /Users/annakessler/Desktop/MA Studium /Patterns and Trends/Patterns

Exercise B

Funktionen aus der Aufgabe A, die wir brauchen:

```
library("readr")
library("sf")
library("dplyr")

difftime_secs <- function(later, now){
  as.numeric(difftime(later, now, units = "secs"))
}

distance_by_element <- function(later, now){
  as.numeric(
    st_distance(later, now, by_element = TRUE)
  )
}
```

Import Data

```
caro <- read_delim("Data/car060.csv", ",", "%>%
  st_as_sf(coords = c("E","N"), crs = 2056) %>%
  select(DatetimeUTC)
```

Rows: 200 Columns: 6

-- Column specification -----

Delimiter: ","

chr (2): TierID, TierName

dbl (3): CollarID, E, N

dtm (1): DatetimeUTC

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Calculate speed at scale 1

Scale ist 120s zwischen den Messzeitpunkten. Jetzt den Speed berechnen mit scale 1 (zwischen den zeitpunkten)


```
now = lag(caro$DatetimeUTC)
later = lead(caro$DatetimeUTC)

now[1:10]
```

```
[1] NA "2015-09-15 08:07:00 UTC"
[3] "2015-09-15 08:08:00 UTC" "2015-09-15 08:09:00 UTC"
[5] "2015-09-15 08:10:00 UTC" "2015-09-15 08:11:00 UTC"
[7] "2015-09-15 08:12:00 UTC" "2015-09-15 08:13:00 UTC"
[9] "2015-09-15 08:14:00 UTC" "2015-09-15 08:15:00 UTC"
```

```
later[1:10]
```

```
[1] "2015-09-15 08:08:00 UTC" "2015-09-15 08:09:00 UTC"
[3] "2015-09-15 08:10:00 UTC" "2015-09-15 08:11:00 UTC"
[5] "2015-09-15 08:12:00 UTC" "2015-09-15 08:13:00 UTC"
[7] "2015-09-15 08:14:00 UTC" "2015-09-15 08:15:00 UTC"
[9] "2015-09-15 08:16:00 UTC" "2015-09-15 08:17:00 UTC"
```

```
caro <- mutate(caro,
               timelag = difftime_secs(later, now))

head(caro)
```

Simple feature collection with 6 features and 2 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570489 ymin: 1205095 xmax: 2570589 ymax: 1205130

Projected CRS: CH1903+ / LV95

A tibble: 6 x 3

	DatetimeUTC	geometry	timelag
	<dtm>	<POINT [m]>	<dbl>
1	2015-09-15 08:07:00	(2570589 1205095)	NA
2	2015-09-15 08:08:00	(2570573 1205096)	120
3	2015-09-15 08:09:00	(2570536 1205099)	120
4	2015-09-15 08:10:00	(2570518 1205115)	120
5	2015-09-15 08:11:00	(2570499 1205130)	120
6	2015-09-15 08:12:00	(2570489 1205130)	120

```
now = lag(caro$geometry)
later = lead(caro$geometry)

now[1:10]
```

```
Geometry set for 10 features (with 1 geometry empty)
Geometry type: POINT
Dimension:      XY
Bounding box:   xmin: 2570481 ymin: 1205095 xmax: 2570589 ymax: 1205130
Projected CRS: CH1903+ / LV95
First 5 geometries:
```

```
POINT EMPTY
```

```
POINT (2570589 1205095)
```

```
POINT (2570573 1205096)
```

```
POINT (2570536 1205099)
```

```
POINT (2570518 1205115)
```

```
later[1:10]
```

```
Geometry set for 10 features
Geometry type: POINT
Dimension:      XY
Bounding box:   xmin: 2570481 ymin: 1205091 xmax: 2570573 ymax: 1205130
Projected CRS: CH1903+ / LV95
First 5 geometries:
```

```
POINT (2570573 1205096)
```

```
POINT (2570536 1205099)
```

```
POINT (2570518 1205115)
```

```
POINT (2570499 1205130)
```

```
POINT (2570489 1205130)
```

```

caro <- mutate(caro,
               steplength = round(distance_by_element(later, now), 1))

caro <- mutate(caro,
               speed = round(steplength/timelag, 3))

head(caro)

```

Simple feature collection with 6 features and 4 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570489 ymin: 1205095 xmax: 2570589 ymax: 1205130

Projected CRS: CH1903+ / LV95

A tibble: 6 x 5

	DatetimeUTC <dtm>	geometry <POINT [m]>	timelag <dbl>	steplength <dbl>	speed <dbl>
1	2015-09-15 08:07:00	(2570589 1205095)	NA	NA	NA
2	2015-09-15 08:08:00	(2570573 1205096)	120	52.4	0.437
3	2015-09-15 08:09:00	(2570536 1205099)	120	58.4	0.487
4	2015-09-15 08:10:00	(2570518 1205115)	120	49.2	0.41
5	2015-09-15 08:11:00	(2570499 1205130)	120	32.6	0.272
6	2015-09-15 08:12:00	(2570489 1205130)	120	18	0.15

Calculate speed at scale 2

Hier nehmen wir 240s scale, um diese zu vergleichen.

- Tip: Use the `n =` parameter in `lead/lag` to increase the offset.
- Store values **timelag**, **steplength** and **speed** in the columns **timelag2**, **steplength2** and **speed2** to distinguish them from the values from scale 1

```
head(caro)
```

Simple feature collection with 6 features and 4 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570489 ymin: 1205095 xmax: 2570589 ymax: 1205130

Projected CRS: CH1903+ / LV95

A tibble: 6 x 5

	DatetimeUTC <dtm>	geometry <POINT [m]>	timelag <dbl>	steplength <dbl>	speed <dbl>
--	----------------------	-------------------------	------------------	---------------------	----------------

1	2015-09-15 08:07:00 (2570589 1205095)	NA	NA	NA
2	2015-09-15 08:08:00 (2570573 1205096)	120	52.4	0.437
3	2015-09-15 08:09:00 (2570536 1205099)	120	58.4	0.487
4	2015-09-15 08:10:00 (2570518 1205115)	120	49.2	0.41
5	2015-09-15 08:11:00 (2570499 1205130)	120	32.6	0.272
6	2015-09-15 08:12:00 (2570489 1205130)	120	18	0.15

```
now = lag(caro$DatetimeUTC, n=2)
later = lead(caro$DatetimeUTC, n=2)

now[1:10]
```

```
[1] NA NA
[3] "2015-09-15 08:07:00 UTC" "2015-09-15 08:08:00 UTC"
[5] "2015-09-15 08:09:00 UTC" "2015-09-15 08:10:00 UTC"
[7] "2015-09-15 08:11:00 UTC" "2015-09-15 08:12:00 UTC"
[9] "2015-09-15 08:13:00 UTC" "2015-09-15 08:14:00 UTC"
```

```
later[1:10]
```

```
[1] "2015-09-15 08:09:00 UTC" "2015-09-15 08:10:00 UTC"
[3] "2015-09-15 08:11:00 UTC" "2015-09-15 08:12:00 UTC"
[5] "2015-09-15 08:13:00 UTC" "2015-09-15 08:14:00 UTC"
[7] "2015-09-15 08:15:00 UTC" "2015-09-15 08:16:00 UTC"
[9] "2015-09-15 08:17:00 UTC" "2015-09-15 08:18:00 UTC"
```

```
caro <- mutate(caro,
               timelag2 = difftime_secs(later, now))

now = lag(caro$geometry, n=2)
later = lead(caro$geometry, n=2)

caro <- mutate(caro,
               steplength2 = round(distance_by_element(later, now), 1))

caro <- mutate(caro,
               speed2 = round(steplength2/timelag2, 3))

head(caro)
```

Simple feature collection with 6 features and 7 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570489 ymin: 1205095 xmax: 2570589 ymax: 1205130

Projected CRS: CH1903+ / LV95

A tibble: 6 x 8

	DatetimeUTC <dtm>	geometry <POINT [m]>	timelag <dbl>	steplength <dbl>	speed <dbl>
1	2015-09-15 08:07:00	(2570589 1205095)	NA	NA	NA
2	2015-09-15 08:08:00	(2570573 1205096)	120	52.4	0.437
3	2015-09-15 08:09:00	(2570536 1205099)	120	58.4	0.487
4	2015-09-15 08:10:00	(2570518 1205115)	120	49.2	0.41
5	2015-09-15 08:11:00	(2570499 1205130)	120	32.6	0.272
6	2015-09-15 08:12:00	(2570489 1205130)	120	18	0.15

i 3 more variables: timelag2 <dbl>, steplength2 <dbl>, speed2 <dbl>

Calculate speed at scale 3

Hier scale 480s

```
now = lag(caro$DatetimeUTC, n=4)
later = lead(caro$DatetimeUTC, n=4)

caro <- mutate(caro,
               timelag3 = difftime_secs(later, now))

now = lag(caro$geometry, n=4)
later = lead(caro$geometry, n=4)

caro <- mutate(caro,
               steplength3 = round(distance_by_element(later, now), 1))

caro <- mutate(caro,
               speed3 = round(steplength3/timelag3, 3))
head(caro)
```

Simple feature collection with 6 features and 10 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2570489 ymin: 1205095 xmax: 2570589 ymax: 1205130

Projected CRS: CH1903+ / LV95

```
# A tibble: 6 x 11
  DatetimeUTC          geometry timelag steplength  speed
  <dtm>          <POINT [m]>   <dbl>    <dbl>    <dbl>
1 2015-09-15 08:07:00 (2570589 1205095)      NA      NA      NA
2 2015-09-15 08:08:00 (2570573 1205096)    120    52.4  0.437
3 2015-09-15 08:09:00 (2570536 1205099)    120    58.4  0.487
4 2015-09-15 08:10:00 (2570518 1205115)    120    49.2  0.41
5 2015-09-15 08:11:00 (2570499 1205130)    120    32.6  0.272
6 2015-09-15 08:12:00 (2570489 1205130)    120     18  0.15
# i 6 more variables: timelag2 <dbl>, steplength2 <dbl>, speed2 <dbl>,
#   timelag3 <dbl>, steplength3 <dbl>, speed3 <dbl>
```

Compare speed across scales

Laube and Purves (2011). In their experiments, the authors observe:

- A steady decrease in median speed as the temporal analysis scale increases;
- A decrease in the overall variance in speed as the temporal scale increases;
- Lower minimum values at the shortest temporal scales;

Wir wollen unsere Daten mit den verschiedenen Skalen nun auch vergleichen. Dafür müssen wir unsere Daten leicht modifizieren.

```
head(caro |>
  st_drop_geometry() |>
  select(DatetimeUTC, speed, speed2, speed3))
```

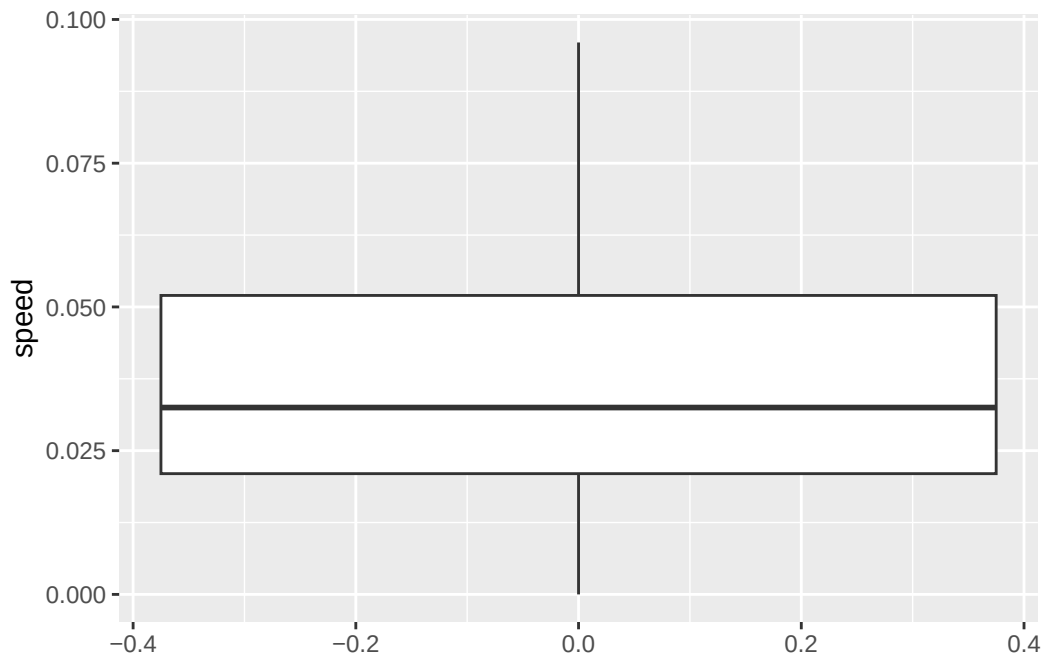
```
# A tibble: 6 x 4
  DatetimeUTC          speed speed2 speed3
  <dtm>          <dbl>   <dbl>   <dbl>
1 2015-09-15 08:07:00  NA      NA      NA
2 2015-09-15 08:08:00  0.437  NA      NA
3 2015-09-15 08:09:00  0.487  0.402  NA
4 2015-09-15 08:10:00  0.41   0.378  NA
5 2015-09-15 08:11:00  0.272  0.248  0.214
6 2015-09-15 08:12:00  0.15   0.155  0.172
```

Boxplot einer einzelnen Spalte im Datensatz:

```
library(ggplot2)

ggplot(caro, aes(y = speed)) +
  # we remove outliers to increase legibility, analogue
  # Laube and Purves (2011)
  geom_boxplot(outliers = FALSE)
```

Warning: Removed 2 rows containing non-finite outside the scale range (`stat\_boxplot()`).



Mit `pivot_longer`, `tidyr` machen wir einen long table, damit wir die Spalten der speeds vergleichen können. Dann machen wir daraus einen fertigen Boxplot, der Vergleiche zulässt.

```
library(tidyr)

# before pivoting, let's simplify our data.frame
caro2 <- caro |>
  st_drop_geometry() |>
  select(DatetimeUTC, speed, speed2, speed3)

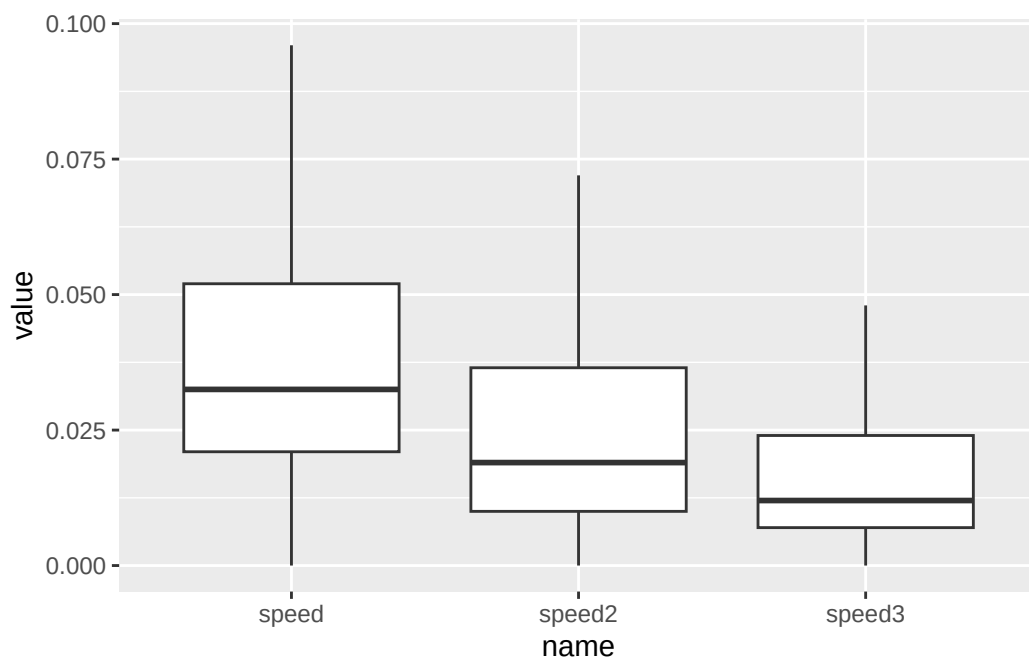
caro_long <- caro2 |>
  pivot_longer(c(speed, speed2, speed3))
```

```
head(caro_long)
```

```
# A tibble: 6 x 3
  DatetimeUTC      name  value
  <dtm>         <chr>  <dbl>
1 2015-09-15 08:07:00 speed   NA
2 2015-09-15 08:07:00 speed2  NA
3 2015-09-15 08:07:00 speed3  NA
4 2015-09-15 08:08:00 speed    0.437
5 2015-09-15 08:08:00 speed2  NA
6 2015-09-15 08:08:00 speed3  NA
```

```
ggplot(caro_long, aes(name, value)) +
  # we remove outliers to increase legibility, analogue
  # Laube and Purves (2011)
  geom_boxplot(outliers = FALSE)
```

Warning: Removed 14 rows containing non-finite outside the scale range (`stat\_boxplot()`).



Man sieht, dass der median speed mit Scale 1 am höchsten war (120s), dann bei scale 2 (240s) und dann scale 3 bei 480s. Auch die Varianz nimmt ab mit zunehmender scale.

WARUM?

- Kleine Zeitintervalle (z. B. 120 s):
 - erfassen kurzfristige Bewegungen
 - enthalten mehr „Spitzen“ (hohe Geschwindigkeiten)
 - auch mehr Messrauschen
- Große Intervalle (z. B. 480 s):
 - mitteln Bewegung über längere Zeit
 - Stopps und langsame Phasen werden mit einbezogen
 - **dadurch sinkt die berechnete Geschwindigkeit**